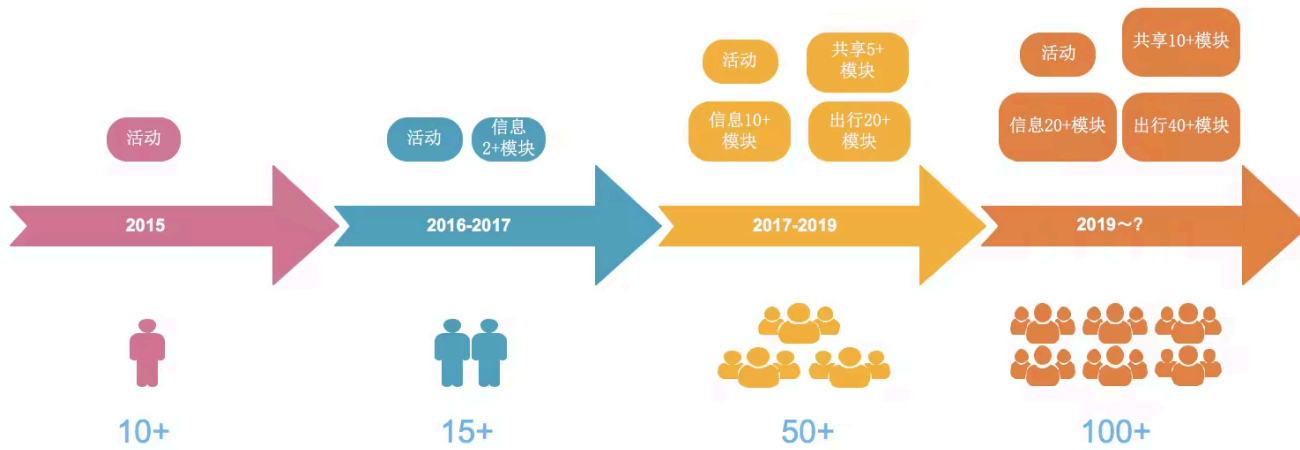


高德前端这五年

2015~2020，经历了5年发展，「应用开发前端团队」在高德业务快速发展中不断成长，一路走来从不到10人负责短期活动缺少技术沉淀的散兵游勇，到现在团队规模100+覆盖高德3大业务线，80多个模块的坚甲利兵。



AMAP前端的发展按照时间线来看，可以分为4个阶段

- 2015 业务上拉新的诉求，活动需求暴增，前端开始登上AMAP大舞台
- 2016~2017 业务高速发展对于效率以及双端一致性的诉求，带来了发展的契机
- 2017~2019 AJX全面落地，前端开发的角色越来越重要，业务半径不断延展
- 2019~? 这是目前的发展阶段，这个阶段我们更关注支撑的稳定性和延展性，让业务更好的活在未来。

一言蔽之“顺势而为，乘风破浪”，接下来让我们一起回顾下。

小荷才露尖尖角（2015）

2014年底，公司提出专注用户需求，专注做最好的地图导航产品，专注地图导航产品的技术研究，三年内无商业目标的“新高德”战略。甩开商业化包袱，一心专注产品的高德地图，就此踏上了高速发展的轨道。

运营活动需求暴增，“工程、效率”解题

围绕公司促进日活和留存的战略，大量运营活动需求应运而生，这个阶段的活动特点是“短平快”开发周期短，一周内交付验证，活动时效期过后即可下线，完全不需要维护。对于代码的可维护性、技术上的创新要求不高，目标是快速响应热点事件，完成活动开发。这种模式给当时不到10人的前端团队带来了非常大的考验，因为很多运营活动搭建需要在前端完

成。而此时的前端团队在技术沉淀上较为薄弱，重复劳动明显，基于此我们首先完成了效率上的提升。





8.20~9.3北京单双号限行

单号出行				双号出行			
							1
2	3	4	5	6	7	8	
9	10	11	12	13	14	15	
16	17	18	19	20	21	22	
23	24	25	26	27	28	29	
30	31	1	2	3			

高德大数据告诉你
这15天出门会有啥变化!





【效率】：完成Aflow工具建设，组件化、模版化、流程化三方面提高活动开发效率。

- 组件化：和运营同学一起规范活动常用组件。
- 模版化：通过模版化解决简单页面的搭建问题。
- 流程化：CLI加速工作流。

正是基础能力上的耕耘，在之后多个热点事件时，我们才能游刃有余在短时间内完成业务开发、上线。

十一全民出行节，“性能、体验”沉淀

时间回到15年10.1，为了配合十一全民出行节，第一个大型运营活动十一挖宝就此诞生，也拉开了每年出行高峰必有大型活动的序幕。更加简单、有趣的交互设计提升用户的游戏体验，强烈的社交属性例如PK，排行榜等促进用户之间传播。

这一年的活动公司进行了大范围的运营推广，“寻宝嘉年华，十六台Smart汽车送不停”，“斗鱼主播全场直播挖宝”小伙伴们应该还有印象，当时直播间非常火热，我们却战战兢兢，如

履薄冰特别担心直播时出现卡顿，白屏等问题由火热的全网挖宝变成全网大型吐槽节目，当时开玩笑说直播出现事故明天就不用来了，由于主播的使用的机器大多性能比较好，我们担心的事情没有发生。

尽管如此，后背发凉的回忆使我们意识到，技术上如何完善复杂游戏的性能体验必将成为日后的课题，基于此我们又完成了基础技术能力上的沉淀

【体验、性能沉淀】

- 音频语音交互解决方案
- 大型游戏，性能最佳实践。



到15年底整个前端团队初具雏形，团队开始建立规范化，标准化，体系化的思想，在技术上也积攒了不少家底

为了应对可预见的考验，前端团队也招入了很多有能力的新人。正是这些人才，使我们在接

下来的多线作战中游刃有余。

忽如一夜春风来（2016-2017）

随着高德地图业务沿着扩品类、在垂直品类做精做细，景区、酒店、银行商铺、充电桩等个性化定制需求凸显，对前端展现提出了更高的要求，对“快速应变”要求也越来越高，当时主要面临以下痛点

- 1、业务要求快速发版试错
- 2、研发资源越来越无法满足业务的快速增长

契机，AJX诞生

这些问题也在不断地督促我们去反思、到底有没有一种架构既能象h5一样快速的开发、发布又能保持原生native的体验？实际上，在2015年，我们就开始做动态化了，最早业内就有ReactNative，团队做了技术调研，发现不能完全满足业务上的需要，尤其是性能方面，因此我们决定自研一套动态化技术，在项目伊始就有一些难点摆在我们面前

- 布局怎么做？
 - **RN的yoga**、iOS的绝对布局、Android的RelativeLayout？
- Runtime framework放在哪里？
 - **C++**、JS谁来承载
- 模块化的机制是什么样的？
 - **Node Require**、Webpack Require
- 通信、动画怎么做种种问题需要我们探索，抉择。

经过团队内部多次思想上的碰撞、激烈的讨论，最终确定**核心设计思想**

- **核心**：核心处理尽量下沉AJX引擎层，双端尽量做薄，AJX引擎（C++）以**Webkit**、**Node为参考**，即可以通过HTML、CSS、JavaScript编写原生应用，又可以像Nodejs一样使用文件操作等与原生应用的交互能力。这样的设计在上层对接**前端生态时更加灵活**，在处理复杂、频繁交互的大型页面时也会有更好的**性能**。
- **优化**：除常规动画外，还设计了关联动画解决**高频联动动画**，关联动画本质上并非是一种播放类型的动效，是基于观察者模式设计的，被观察者的属性变化会影响观察者的属性变化，它将关联关系提前绑定好，一次性由「js 线程」传递给「UI 线程」，这样能够很好的保证交互性能。

再方案明确后，整个团队也投入到能力建设中来，尽管每周都在发现问题、解决问题中艰难前行大家仍然乐此不疲，对于这种打怪升级的过程乐在其中。在基础能力、辅助工具齐备的背景下，此时的我们开始着手AJX的落地实践，最终我们选择了POI业务。

起航，AJX落地POI



首先看一看POI业务的特点

- UI复杂，多品类，多种多样的展现形式
- 与地图有存在交互
- 性能要求高，长列表，数据量大
- 富交互，大量手势交互，关联动画



北京方恒假日酒店

距您 782 米 朝阳区阜通东大街 6 号院 3 号楼

高档型  高端酒店排行 No.3 >



电话

🕒 03月19日 今天入住 - 03月20日 明天离店 共1晚 >

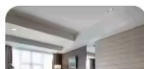
含早

大床房

双床房

信用住

筛选 ▾



高级房

33m² 无早 大/双床 有窗 31-35层

🔍 搜周边

📍 导航

📍 路线



北京方恒假日酒店



酒店预订 住客评价 设施/详情 热门推荐

含早

大床房

双床房

信用住

筛选 ▾

33m² 无早 大/双床 有窗 38-41层

03月27日18点前可免费取消

去哪儿

¥1206
¥1107
已减99元

预订

在线付

33m² 无早 大床 有窗 38-41层

03月27日18点前可免费取消

去哪儿

¥1206
¥1107
已减99元

预订

在线付

33m² 无早 大/双床 有窗 38-41层

入住当天18点前可免费取消

艺龙

¥1126

预订

在线付

33m² 双早 大/双床 有窗 38-41层

免费取消

🔍 搜周边

📍 导航

📍 路线

住客评价

写评论

服务周到 275 房子不错 237 交通便利 145

早餐好 79 设施高大上 79 景色好 53

回头客 51 靠近机场 23 泳池不错 17



_M1361113****

★★★★★

酒店早餐中餐的话很好，好像是偏商务的酒店。离地铁站近，购物方便。双床的房间床是1.4米宽，带孩子很好；大床的话可以加床，比较贵。没打满分是...
全文

2020-01-05 携程网



amap_137****1602lf3ZLiOR

★★★★★

酒店大堂面朝庭院，静谧清幽，既是繁华都市的一道亮丽风景线，亦是置身于喧闹都市中的世外桃源。酒店拥有275间全景观客房其中包括61间行政客房和2...

搜周边

导航

路线

POI完全满足我们对基础能力验证的目标，并且POI线上有成熟的h5页面，即使出现问题也可以立即切换h5进行更新发布，影响最小。但要使项目平稳落地，我们仍然需要解决一些问题

- 多人协作开发问题：为了快速验证能力，项目的排期非常紧张，为此前端同学All in，业务上看尽管POI只有一个页面，但是却有多个行业，而行业是由多个模块拼接而成，每个模块在不同行业展现形式也不尽相同，那么如何解决协作问题就成为项目成败的关键之一
 - 为此我们完成了 [framework框架](#)，可以用JSX语法实现基本组件化，在组件这个级别进行「CURD」解决了模块化开发的问题
- 调试问题：在项目之初我们并不存在完备的调试方式，甚至可以说不存在调试能力，只是通过print将log输出到手机端展现。这在开发POI时遇到了极大的问题，业务场景复杂大量实时日志无法查看，导致效率极低。
 - 为此我们完成了websdk，mock能力，在浏览器端完成了POI页面的预览，调试。

尽管POI落地过程中，遇到了各种各样的问题，但结果是美好的，AJX也经受住了业务的考验，新的POI不仅完全覆盖了之前h5的功能，在手势动画、list展现上还体现了更加卓越的交互体验和性能。伴随着业务上线，AJX基建一期也基本完成，这个阶段以满足业务为中心，主要围绕支撑能力的设计和基本的开发体验。



坦白讲，当时的支撑能力并不完善，在开发过程中有着各种各样的痛苦，但在业务验证前的基础建设是困难的，难的不是方案、而是执行。POI的圆满落地也标志着前端技术有能力在AMAP中承担更复杂、更核心、更大的业务场景，前端开发也即将迎来春天。

千树万树梨花开（2017~2019）

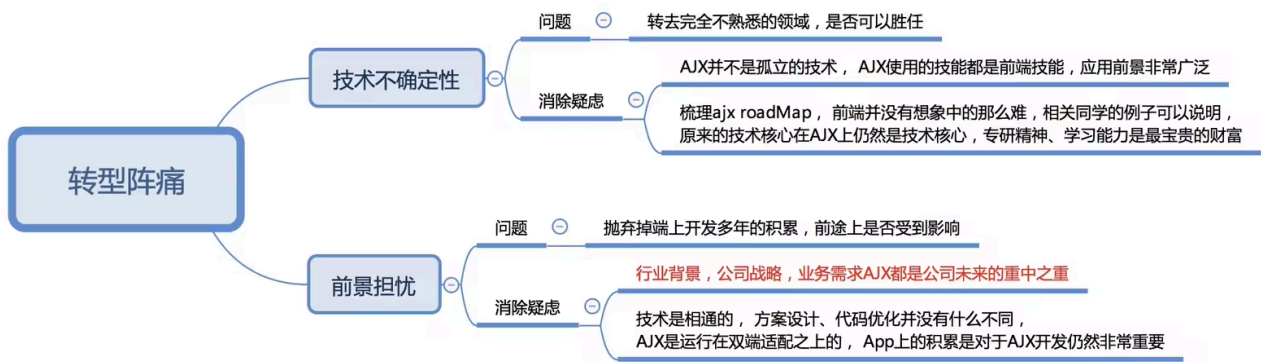
POI业务上的成功落地，标志着AJX解决方案趋于稳定，可以应对各种各样的复杂业务，随之而来的是大量业务的考验。随着AJX应用的深入、主要业务模块的全面接入，支撑能力不完善、AJX开发人员缺乏导致改造压力越来越大。

团队壮大，“小前端”到“大前端”

唯一不变的是变化

人员的问题主要从内外两方面解决，外部启动招聘专项，大量吸纳有相关背景的前端同学。内部Native同学加强技术培训，部分转向AJX开发也正式提上日程。

不少同学一定有这样的经历，如果让其去调研一门新的技术大家一定非常乐意，充满干劲，对未知领域的探索，求知渴望是研发的共同点。不过如果让其持续朝着这门技术发展就会有非常多的疑虑。



通过培训等方式，解决大家对于技术不确定性和前景的担忧，大量同学开始转向AJX开发，到19年初整个“大前端”从15+发展到100+团队，其中Native同学占比超过50%。

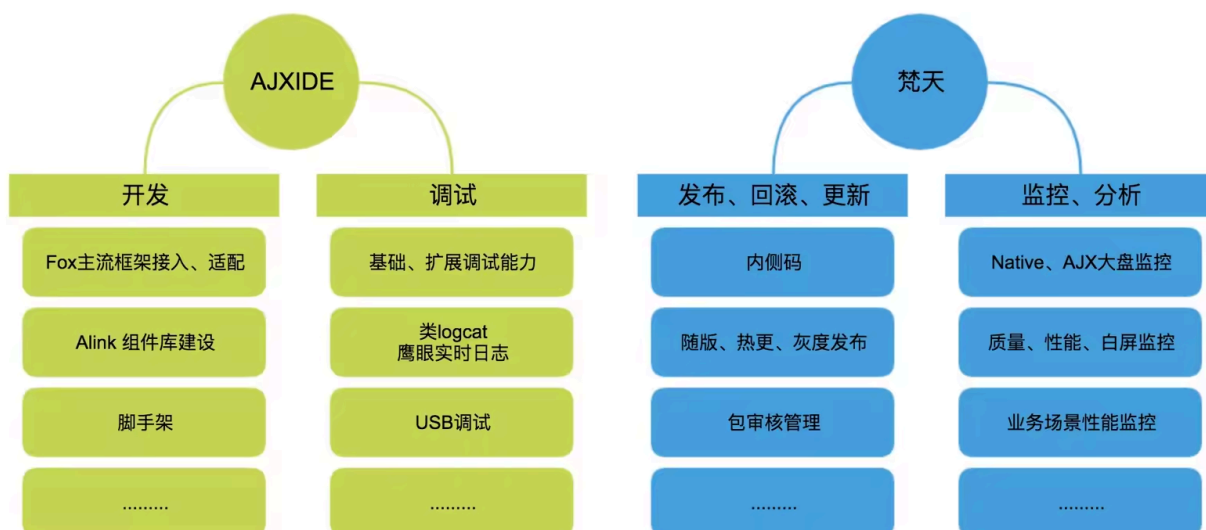
基建完善，“研发闭环，逐个突破”



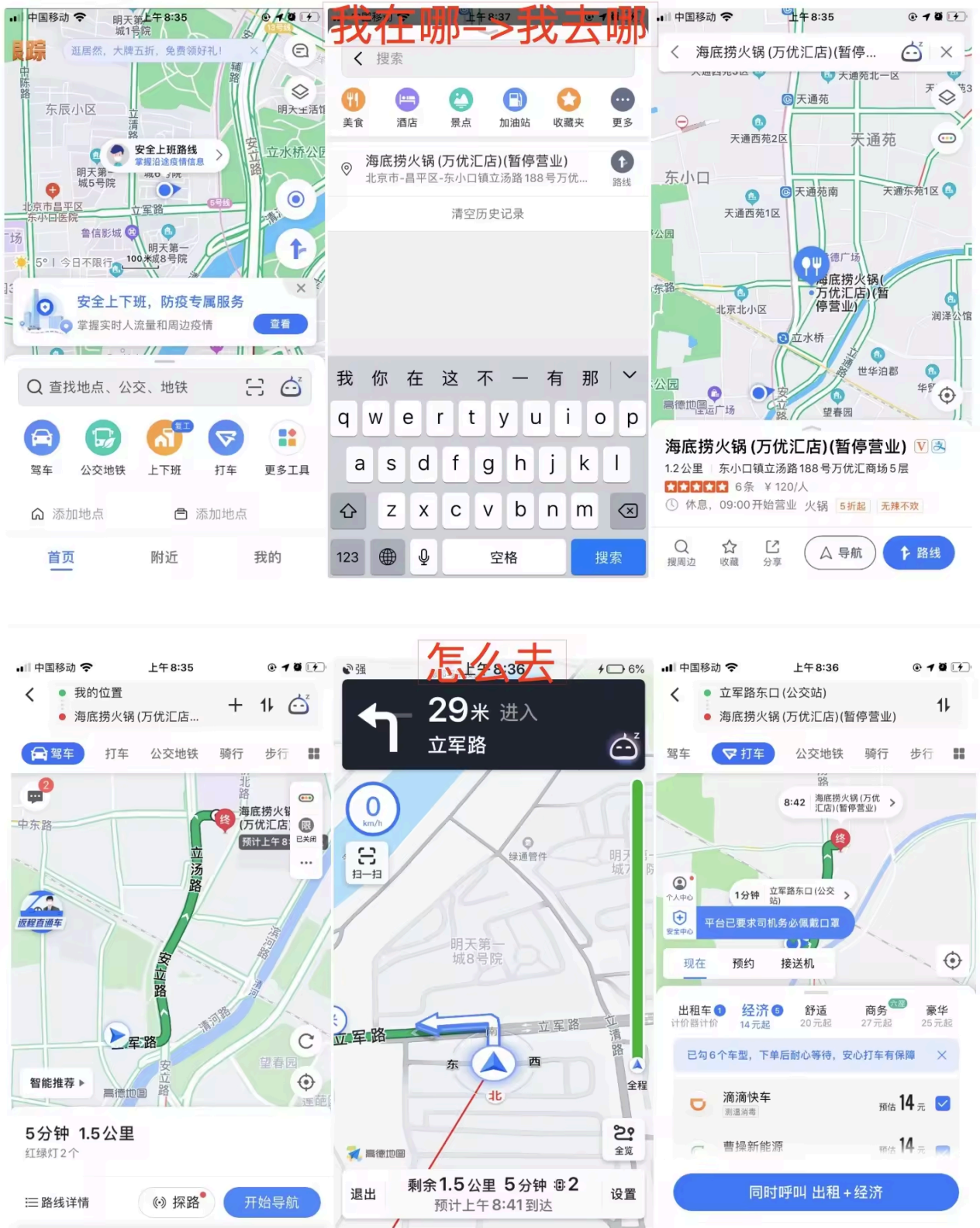
研发队伍的扩大使我们对于业务支持和改造信心十足，但研发效率渐渐成为了业务发展的瓶颈，以业务提效为主题的基建工作全面展开，我们从研发流程入手，针对流程闭环按优先级逐个突破

在此期间我们完成了

- 通过AJXIDE将开发、调试能力打通
- 通过梵天平台进行发布、回滚、更新、监控、分析操作



业务增长，“横跨3大业务线，Amap核心业务全覆盖”



富有战斗力的团队、渐渐完善的基建使业务从小步慢走到大步快跑，从用户的核心诉求我在哪（主图，定位），我去哪（搜索，POI），怎么去（规划，导航）到用户的延伸诉求怎么去（打车）怎么玩（景区）高德3大业务线60多个模块全面接入AJX技术。



这个阶段AJX业务发展迅猛，“前端团队”不断壮大，由“小前端”转变为“大前端”，基建方面也是围绕业务全面展开不断完善，有了稳定的开发环境，17~18不到2年的时间我们完成代码量从3w到60w，模块数量1到60，开发人员15+到100+的井喷式增长。业务发版频次渐渐加快、加密，从「单月版」→「快迭双周版」

九层之台，起于垒土（2019-？）

面对着越来越复杂的业务，细节问题不断暴露，回头看去如何更好的为业务赋能再次成为重点，

玉伯之前分享有个词印象颇深，愿等花开：坚持长期主义，要快，但不能急。

回到自身来说：

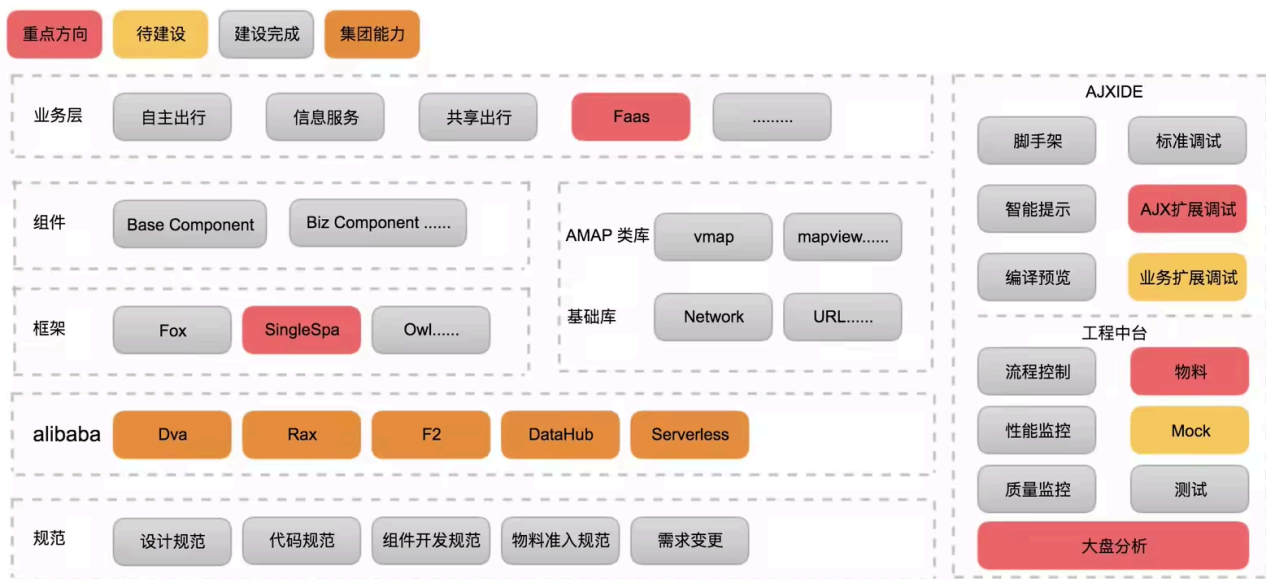
- 前面几年都是保证业务赢在当下，支撑上都是大刀阔斧快速建设，完成0到1的过程。
- 接下来应该帮助业务更好的活在未来，在当前基础能力具备的情况下，需要闭关审视自身，从功能的完整性，延展性等方面做到精细化。

未来我们也将从研发生态、工程中台、跨端技术3大方向上以精细化、标准化、差异化为基础要求不断补足能力，渐渐完成中台化、智能化的基础建设工作，围绕AJXIDE打造更好的一站式场景化开发体验。

- 五化基建方针：
 - 精细化：切中痛点，系统解题
 - 标准化：集团生态、业界标准
 - 差异化：在标准化的基础上，提供AJX、业务能力扩展。
 - 智能化：低代码，物料复用，UI自动生成

- 中台化：前台通用能力下沉到中台，不再局限Amap

技术大图：



接下来重点方向

- 工具链路稳定性、延展性持续优化
- 平台能力中台化
- Faas：前端承载更多、更了解业务
- 物料：缩短开发到资源路径，沉淀更多基础能力
- 多框架融合：解决老代码维护问题